

Combinadores de Interação para Computação em Grade: Avaliação Experimental de Redução Distribuída com Confluência Forte

Filipe Andrade Nascimento
Yuri Faro Dantas de Sant Anna (Orientador)
Universidade Tiradentes

Resumo—A computação em grade tem como objetivo distribuir o trabalho por redes de longa distância, onde a latência varia e partes do sistema podem falhar sem aviso. Nesse cenário, um modelo de computação cujo resultado é determinístico por construção é um candidato natural. Este trabalho investiga se os combinadores de interação (CI) de Lafont, que computam por reescrita local de um grafo, podem cumprir esse papel. Sua propriedade central é a confluência forte, em que as regras, aplicadas em qualquer ordem, levam à mesma rede final. Trabalhos anteriores exploraram essa propriedade apenas em memória compartilhada, não em uma grade. A hipótese central deste trabalho é que a confluência forte, combinada a um protocolo que preserva a estrutura da rede, faz a redução distribuída de uma rede terminante convergir à mesma forma normal da redução sequencial, qualquer que seja o número de *workers*. O Relativist foi construído para verificar esta hipótese. O modelo proposto é um redutor distribuído, implementado em Rust e desenvolvido sob *Spec-Driven Development*, com 28 especificações formais e cerca de 800 requisitos rastreáveis. Três campanhas o exercitaram: execução local, comunicação TCP em *localhost* via contêineres e uma varredura que cobre quatro ordens de grandeza no tamanho da rede. Em cerca de 5350 reduções, a forma normal distribuída coincidiu com a sequencial em todos os casos; uma análise de *break-even* indica sob que largura de banda e custo por interação a distribuição passa a ser vantajosa. Até o presente momento não foi verificada outra validação empírica dessa equivalência em uma arquitetura de grade. O resultado sustenta o uso de modelos com determinismo estrutural em sistemas distribuídos sujeitos a falhas.

Index Terms—interaction combinators, grid computing, confluência forte, redução distribuída, reescrita de grafos

1 INTRODUÇÃO

AMBIENTES de computação em grade coordenam recursos computacionais espalhados por organizações virtuais que se formam e se desfazem dinamicamente, na definição clássica de Foster, Kesselman e Tuecke [1]. Quando se comunicam por redes de longa distância, esses recursos podem apresentar latência alta e variável, perda e atraso de mensagens e ausência de controle centralizado [2]. Nestes cenários, os recursos executam processos em ordens imprevisíveis e, desta forma, manter o estado consistente exige coordenação explícita. Esse cenário motiva um modelo de computação cujo resultado seja determinístico por construção, sem coordenação global.

Os combinadores de interação (CI) [3] são um modelo de computação mínimo: três símbolos e seis regras que reescrevem um grafo. Duas propriedades importam para este trabalho. A primeira é a *confluência forte*, em que as regras, aplicadas em qualquer ordem, produzem o mesmo resultado final. A segunda é a *localidade*, em que cada passo de computação afeta apenas os dois agentes que interagem. Apesar da simplicidade, o sistema é *universal*, Lafont [3] prova que qualquer computação expressável em *interaction nets* [4] pode ser traduzida para ele. A Seção 2.1 formaliza esses conceitos.

Até o presente momento, os trabalhos que se propuseram a explorá-las o fizeram em memória compartilhada. O HVM2 [5] é um avaliador massivamente paralelo baseado em CI estendidos. Ele obteve ganho de velocidade quase proporcional ao número de núcleos. No entanto, o sistema

depende de memória compartilhada. Mackie [6] propôs, em 1997, repartir *interaction nets* entre processadores de memória distribuída, mas a proposta não chegou a ser avaliada em ambiente de grade. As propriedades que sustentam o paralelismo em memória compartilhada permanecem, assim, sem teste em computação em grade.

O objetivo deste trabalho é avaliar experimentalmente se a confluência forte e a localidade dos CI, apoiadas por um protocolo que preserva a estrutura da rede, bastam para construir uma redução distribuída em grade cujo resultado coincide com o da redução sequencial, para qualquer número de *workers*, desde que a rede termine. A hipótese é afirmativa, por duas razões apontadas pela literatura. O HVM2 [5] mostrou que essas propriedades sustentam paralelismo real em memória compartilhada. E a confluência forte, por independer da ordem e do local em que cada par ativo é reduzido, não pressupõe memória compartilhada nem topologia física específica. Este trabalho testa a hipótese empiricamente.

Mais precisamente, a hipótese é que a confluência forte, somada a quatro garantias do protocolo, leva à mesma forma normal da redução sequencial, com qualquer número de *workers* e em qualquer ordem de trabalho. As quatro garantias são particionar sem perder conexões, recompor corretamente as fronteiras, atribuir identificadores sem colisão e terminar em número finito de rodadas. A Seção 4 enuncia formalmente essas premissas. Para testá-la, foi cons-

truído o Relativist¹, um sistema em Rust desenvolvido sob *Spec-Driven Development* (28 especificações e cerca de 800 requisitos formais). A cada execução, o sistema verifica se $\text{reduce_all}(\text{net}) \cong \text{run_grid}(\text{net}, n)$, isto é, se as duas reduções produzem grafos isomorfos. A contribuição deste trabalho é demonstrar essa equivalência com dados experimentais.

O restante do artigo organiza-se como segue. A Seção 2 apresenta o referencial teórico. A Seção 3 discute os trabalhos relacionados. A Seção 4 descreve a metodologia. A Seção 5 apresenta os resultados e discussões. A Seção 6 conclui o artigo e aponta trabalhos futuros.

2 REFERENCIAL TEÓRICO

Esta seção trata apenas do modelo formal dos CI e das propriedades que o tornam um candidato à redução distribuída. As implementações e as tentativas anteriores de explorá-lo, em memória compartilhada ou distribuída, são analisadas na Seção 3.

2.1 O Modelo Formal: das Interaction Nets aos Interaction Combinators

Lafont [4] propôs as *interaction nets* em 1990. Trata-se de um modelo de computação baseado em reescrita de grafos. Os vértices do grafo são os *agentes*, cada um rotulado e representando uma operação. Todo agente tem um número fixo de *portas*. Uma delas, a *porta principal*, é a única pela qual o agente interage; as restantes são *portas auxiliares*, por onde entram e saem dados. Dois agentes conectados por suas portas principais formam um *par ativo* (*active pair*). Esse par é a menor unidade de computação, equivalente a uma instrução pronta para executar. Reduzir é aplicar ao par a regra de reescrita correspondente, que o substitui por uma nova configuração de agentes e conexões.

Três restrições sobre as regras de reescrita tornam o modelo adequado à computação distribuída. A primeira, a *linearidade*, obriga cada variável a aparecer exatamente uma vez de cada lado da regra, de modo que nada é duplicado nem descartado implicitamente. A segunda, a *interação binária*, só permite interação entre dois agentes ligados por suas portas principais. A terceira, a *ausência de ambiguidade*, admite no máximo uma regra para cada par de símbolos. Na versão original de 1990 [4], a terceira condição proibia um símbolo de interagir consigo mesmo. Lafont removeu essa proibição em 1997 [3, nota 1], permitindo as interações *gamma-gamma*, *delta-delta* e *epsilon-epsilon*, que são a base dos CI.

Em conjunto, essas três condições garantem a *confluência forte*. Se uma rede N pode ser reduzida de duas maneiras diferentes, ambas levam ao mesmo resultado. Em termos formais, se N se reduz em um passo tanto a P quanto a Q , com $P \neq Q$, então ambos se reduzem, também em um passo, a uma mesma rede R [4, Proposição 1]. Assim, os pares ativos são independentes entre si e podem ser reduzidos em qualquer ordem, por qualquer processador, sem interferência mútua [4]. A *localidade* reforça essa independência,

pois cada interação afeta apenas os dois agentes do par e suas portas imediatas, sem alterar pontos distantes da rede. Juntas, as duas propriedades permitem, em princípio, reduzir todos os pares ativos simultaneamente. Lafont propôs ainda uma disciplina de tipos que evita *deadlock* em redes bem tipadas [4, Proposições 5–7], aspecto que este artigo não aprofunda.

Em 1997, Lafont [3] reduziu o modelo ao mínimo. Os CI são um caso particular de *interaction nets* com apenas três símbolos e seis regras (Fig. 1). Os símbolos são *gamma* e *delta*, ambos com duas portas auxiliares, e *epsilon*, sem nenhuma. Metade trata de símbolos iguais e chama-se *aniquilação*. A regra *gamma-gamma* religa as portas auxiliares de forma cruzada (a porta 1 de um vai à porta 2 do outro), *delta-delta* as religa em paralelo (porta 1 com porta 1, porta 2 com porta 2) e *epsilon-epsilon* apaga os dois agentes. A outra metade trata de símbolos diferentes e chama-se *comutação*. A regra *gamma-delta* substitui o par por quatro novos agentes religados às portas originais; *gamma-epsilon* e *delta-epsilon* apagam o agente de duas portas e deixam dois *epsilon*s em seu lugar.

As duas aniquilações não são iguais. É *gamma-gamma*, e não *delta-delta*, que troca as portas auxiliares [3, p. 82]. A distinção importa, pois Lafont mostra que ambas as regras são necessárias para a universalidade do sistema [3, p. 90]. O resultado central é que qualquer sistema de *interaction nets* pode ser traduzido para esses três combinadores sem perder complexidade nem paralelismo (Teorema 1 e Proposição 4 de [3]). A computação prossegue enquanto existir par ativo; quando não resta nenhum, a rede atingiu sua *forma normal*, que é o resultado da computação.

Lafont mostrou também que remover qualquer uma das regras (salvo *delta-epsilon*) quebra a universalidade, e conjecturou que não há sistema mais simples: “*we conjecture that our system of interaction combinators is essentially the simplest one satisfying Theorem 1*” [3, p. 90]. Mackie [6] observou que esses combinadores têm, para as *interaction nets*, papel análogo ao dos combinadores S e K na lógica combinatória.

Trabalhos posteriores consolidaram a teoria dos CI. Mackie e Pinto [7] mostraram que os principais fragmentos da lógica linear, a saber, o multiplicativo, o exponencial e o aditivo, podem ser codificados neles, ainda que nem todos os casos sejam cobertos. Mazza e Ross [8] analisaram a variante simétrica do sistema (de mesmo poder de expressão) e provaram que a confluência forte induz uma noção precisa de equivalência semântica; com isso, reafirmaram as *interaction nets* como “um modelo de computação distribuída e determinística” [8, p. 316]. Lafont resume: o tempo de computação é “relativista”, pois dispensa relógio global [3].

3 TRABALHOS RELACIONADOS

Esta seção examina as implementações e as tentativas anteriores de explorar as propriedades dos CI, da memória compartilhada à distribuída, e o que cada uma revelou sobre distribuir a redução.

O HVM2 [5] constitui a principal evidência empírica de que as propriedades dos CI habilitam paralelismo real. Trata-se de um avaliador massivamente paralelo que compila programas de alto nível para C e CUDA, usando CI estendidos (com agentes adicionais REF, NUM, OPE e

1. Relativist: redutor distribuído de CI em Rust, de código aberto. Release v0.20.0-pre, disponível em <https://github.com/andrade-filipe/relativist>. As campanhas experimentais e seus manifestos de proveniência (com somas SHA-256) acompanham o repositório.

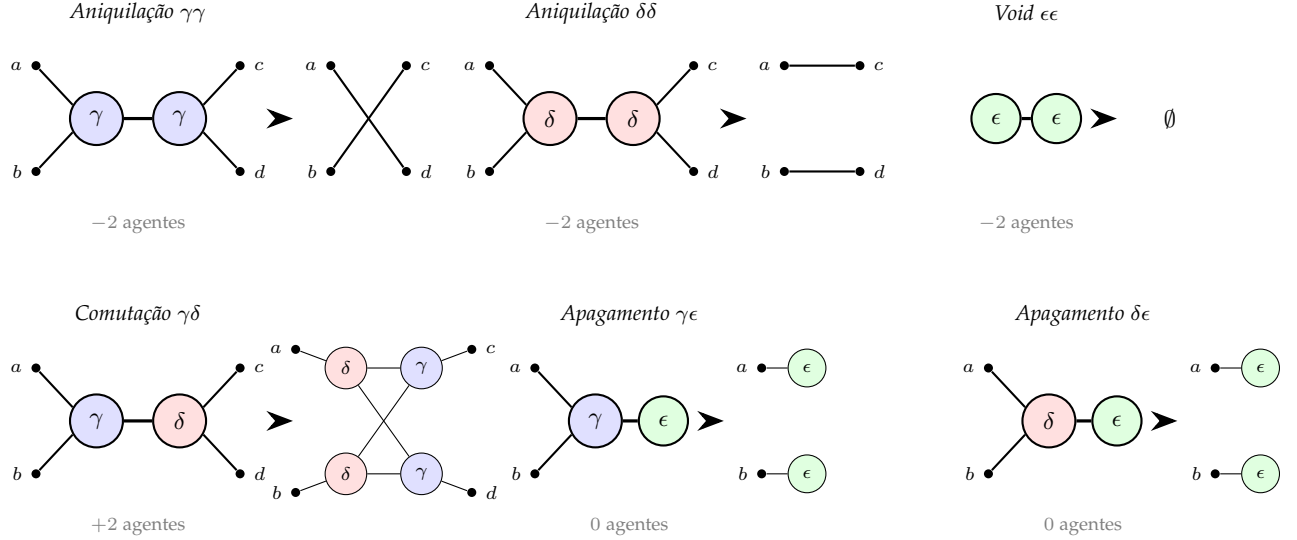


Fig. 1: As seis regras de interação dos CI. Terminais a, b, c, d indicam conexões externas preservadas; a linha grossa indica a conexão entre portas principais (par ativo). Linha superior, aniquilações: $\gamma\gamma$ reconecta cruzado ($a-d, b-c$), $\delta\delta$ reconecta paralelo ($a-c, b-d$), $\epsilon\epsilon$ elimina ambos. Linha inferior: comutação $\gamma\delta$ (+2 agentes, rede bipartida $K_{2,2}$) e apagamentos $\gamma\epsilon$, $\delta\epsilon$ (0 agentes). Ambas as aniquilações binárias são necessárias para a universalidade [3]. Cores: azul = γ , vermelho = δ , verde = ϵ . Diagrama elaborado pelo autor em TikZ com auxílio de IA.

SWI para desempenho prático) como representação intermediária. Cada agente ocupa 64 bits, permitindo interações atômicas sem *locks*. Em CPUs, o sistema usa *task-stealing* (redistribuição dinâmica de pares ativos entre *threads*); em GPUs, anotações de compartilhamento evitam divergência entre *threads* de um mesmo *warp* (grupo de 32 *threads* NVIDIA que executam a mesma instrução simultaneamente). Resultados preliminares reportados por Taelin [5] indicam até 74.000 MIPS em NVIDIA RTX 4090 com 32.768 *threads* e *speedup* quase linear. Esses números provêm de um pré-print cuja seção de *benchmarks* detalhados ainda não foi publicada; devem ser tratados como indicativos. O HVM2, assim como o presente trabalho, opera sobre programas terminantes. Essa restrição é formalizada como premissa P6 na Seção 4.

Contudo, o HVM2 opera exclusivamente em memória compartilhada, em que todas as *threads* acessam um mesmo *buffer* de agentes. Em memória distribuída, onde *workers* se comunicam por rede, surgem questões que a localidade sozinha não resolve: como particionar a rede entre processadores? Como resolver pares ativos cujos agentes residem em recursos diferentes? A confluência forte garante determinismo do resultado, mas não diz nada sobre o custo de uma distribuição mal feita.

Mackie [6] propôs uma técnica de análise estática, apoiada em interpretação abstrata, para repartir *interaction nets* entre processadores. Duas lições resultam dela: o particionamento da rede afeta fortemente o desempenho, e é possível prever estaticamente, de forma aproximada, quais agentes interagirão (o caso geral é indecidível). À época, o próprio autor admitia que não existia “nenhuma implementação distribuída ou paralela bem-sucedida” [6]. Em memória compartilhada, Kahl [9] obteve *speedups* de até $3,8\times$ em 9 núcleos com uma implementação paralela em Haskell, mas não estendeu o trabalho à memória distribuída.

No plano formal, Pinto [10] construiu a primeira

máquina abstrata para reduzir *interaction nets*, decompondo cada interação em operações atômicas sobre uma configuração de *Heap*, *Stack* e *Involution*, e provou que tanto a versão sequencial quanto a concorrente (com *threads* POSIX) estão corretas. A lição relevante para este trabalho é que a confluência forte basta para garantir correteza concorrente, desde que acompanhada de um protocolo de sincronização adequado. Esse é o princípio do protocolo BSP deste trabalho (Seção 4). A máquina de Pinto, porém, restringe-se à memória compartilhada. Mackie e Sato [11] investigaram quais formatos de *interaction nets* se beneficiam da avaliação paralela. Eles separam dois tipos de operação: as *batch*, que só entregam resultado depois de consumir todos os dados, e as *streaming*, que devolvem resultados parciais e permitem mais paralelismo. A conclusão é que o formato do *workload* determina quanto paralelismo é possível extrair. No Fibonacci com adição *streaming*, por exemplo, a complexidade paralela cai de exponencial para quadrática. Essa ideia guia os três perfis de *benchmark* da Seção 4: o Perfil A reúne *workloads batch* puros, o Perfil B mistura *batch* e *streaming*, e o Perfil C cobre o pior caso.

No extremo oposto está a computação em grade sem garantia formal de confluência. Wang et al. [12] registraram o custo de engenharia dessa ausência ao descrever o DDGrid, uma grade para descoberta de fármacos. O sistema precisou de uma arquitetura SEDA (*Staged Event-Driven Architecture*), de detecção de falhas por processamento complexo de eventos, de execução de *backup* para os *workers* mais lentos e do gerenciamento de mais de 1.000 tarefas simultâneas. O contraste é direto. O DDGrid obtém consistência por infraestrutura, enquanto o modelo aqui adotado obtém determinismo como consequência formal da confluência forte. Os demais problemas tratados pelo DDGrid ficam fora do escopo deste trabalho: tolerância a falhas, *stragglers* e escala com milhares de tarefas.

Arrighi et al. [13] mostraram, por fim, que a confluência sozinha não garante determinismo quando a computação distribuída não termina. Esse alerta origina a premissa P6 (Seção 4).

Em síntese, os trabalhos anteriores validaram as propriedades dos CI em memória compartilhada (HVM2 [5], Kahl [9], Pinto [10]) e demonstraram teoricamente a viabilidade de distribuição (Mackie [6]), mas nenhum combinou CI com computação em grade. Este trabalho posiciona-se nessa interseção.

4 METODOLOGIA

A Figura 2 resume o processo completo deste trabalho, da teoria formal à verificação experimental. As propriedades dos CI motivam um conjunto de premissas; estas são formalizadas sob *Spec-Driven Development*, implementadas no Relativist e exercitadas por três campanhas que verificam, a cada execução, a equivalência entre a redução sequencial e a distribuída. As subseções seguintes detalham cada etapa nessa ordem.

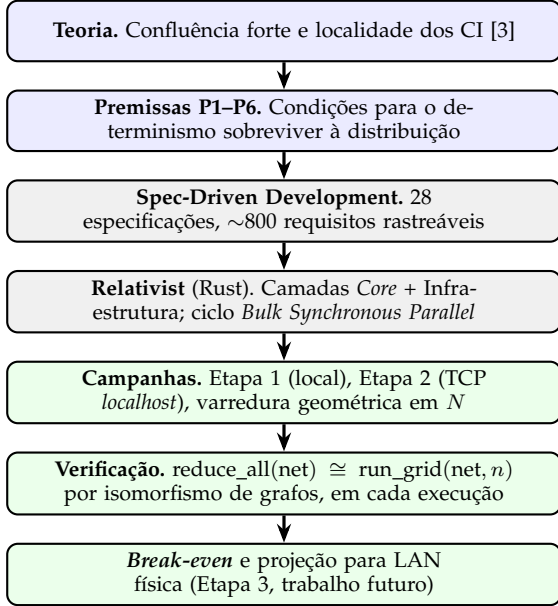


Fig. 2: Visão geral do processo, da teoria ao resultado. As propriedades formais dos CI motivam as premissas P1–P6; estas são formalizadas sob *Spec-Driven Development* e implementadas no Relativist; três campanhas verificam, em cada execução, a equivalência entre a redução sequencial e a distribuída por isomorfismo de grafos; a análise de *break-even* projeta a faixa de banda de rede em que distribuir passa a ser vantajoso. Diagrama elaborado pelo autor em TikZ com auxílio de IA.

4.1 Abordagem: Spec-Driven Development

A investigação adotou *Spec-Driven Development*, de modo que todas as decisões de design foram formalizadas em especificações antes da implementação. A análise sistemática da literatura revelou que a confluência forte é necessária, mas não suficiente, para garantir determinismo em memória distribuída. Seis premissas foram identificadas:

- **P1 — Confluência forte do modelo.** Garantida por construção (Seção 2.1).
- **P2 — Corretude do particionamento.** Cada agente pertence a exatamente uma partição, sem perda de conexões.
- **P3 — Completude da resolução de fronteira.** *Wires* cortados pelo particionamento são restaurados integralmente após a redução local.
- **P4 — Consistência de identificadores.** *Workers* criam novos agentes sem colisão de IDs.
- **P5 — Terminação do protocolo.** O ciclo de redução distribuída converge em número finito de rodadas.
- **P6 — Restrição a redes terminantes.** A rede de entrada atinge forma normal em número finito de interações. Arrighi et al. [13] demonstraram que a confluência sozinha é insuficiente para garantir determinismo em computações não-terminantes distribuídas.

A tese central afirma que, satisfeitas as seis premissas, a redução distribuída produz resultado estruturalmente idêntico ao da redução sequencial. Para torná-la verificável, cada premissa foi associada a um invariante concreto nas especificações do Relativist: a confluência forte, aos teoremas do núcleo de redução; o particionamento, à identidade de *round-trip* entre partição e *merge* e à equivalência da redução local; a fronteira, à detecção dos redexes que cruzam partições no *merge*; os identificadores, à alocação de intervalos disjuntos por *worker*; a terminação, ao progresso monotônico do ciclo BSP. A restrição a redes terminantes delimita o escopo de validade de todos os anteriores.

Todas as premissas foram traduzidas em especificações técnicas verificáveis. A v1 do Relativist reuniu 17 especificações (cerca de 560 requisitos); a v2 (v0.20.0-pre) ampliou esse conjunto para 28 especificações (cerca de 800 requisitos), acrescentando abstração de transporte, formato de transmissão versionado, protocolo *delta*, geração *streaming*, gerência de arena e admissão dinâmica de *workers* (Seção 4.6). O trabalho analítico contou com apoio de ferramentas de inteligência artificial, usadas como assistente de análise crítica sob direção do autor. Por manter as especificações separadas da implementação em Rust, foi possível escrever esta seção antes mesmo de o código estar pronto.

4.2 Arquitetura do Relativist

O Relativist é implementado em Rust e segue o modelo de programação *Bulk Synchronous Parallel* (BSP): cada rodada de redução distribuída corresponde a um *superstep* BSP composto por computação local, comunicação e barreira de sincronização. A arquitetura organiza-se em duas camadas com responsabilidades distintas.

A **camada Core** contém a lógica pura e síncrona, sem dependência de *runtime* assíncrono, rede ou I/O:

A **camada Infrastructure** implementa a comunicação assíncrona sobre *tokio* (*runtime* de I/O assíncrono para Rust): Cada módulo é formalmente especificado.

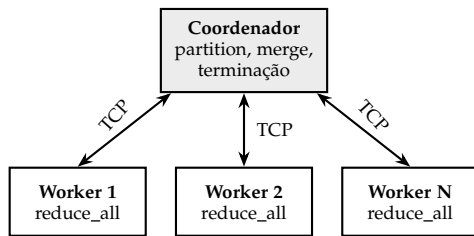
A topologia escolhida é a estrela (Fig. 3). Resolver os *redexes* de fronteira cabe unicamente ao coordenador, que executa *reduce_all* sobre a rede já recombinada. Esse é o único momento em que todos os agentes estão reunidos e

Tabela 1: Módulos da camada Core

Módulo	Responsabilidade
net	Representação da rede: agentes, portas, <i>wires</i> , arena de alocação
reduction	Motor de redução: 6 regras de interação, despacho, <i>reduce_all</i>
partition	Particionamento: função de alocação, <i>FreePort</i> , espaço de IDs
merge	Recombinação: união de partições, resolução de fronteira
encoding	Codificadores de cargas estruturadas (avaliação polinomial pelo método de Horner) e primitivos aritméticos de Church utilizados internamente

Tabela 2: Módulos da camada Infrastructure

Módulo	Responsabilidade
protocol	Protocolo TCP: mensagens, <i>framing</i> binário, serialização
config	Configuração: argumentos CLI, <i>wiring</i> de módulos
security	Autenticação por <i>token</i> e TLS opcional
observability	<i>Tracing</i> estruturado e métricas Prometheus opcionais
io	Formatos de rede, geradores de <i>workloads</i> , exemplos
coordinator	Coordenador: FSM, ciclo BSP, <i>merge</i> centralizado
worker	<i>Worker</i> : recepção, redução local, retorno de resultados



Ciclo BSP: *partition* → *dispatch* → *reduce_local*
→ *collect* → *merge* → *check*

Fig. 3: Topologia estrela do Relativist. O coordenador centraliza particionamento, merge e detecção de terminação. Cada *worker* executa redução local de forma independente, conectando-se diretamente ao coordenador via TCP persistente, sem comunicação *worker-a-worker*. Diagrama elaborado pelo autor em TikZ com auxílio de IA.

os sentinelas *FreePort*(*bid*) foram resolvidos. A decisão simplifica o protocolo e atende à validação experimental proposta, mas limita a escalabilidade. A solução vale enquanto o coordenador não se torna gargalo.

4.3 Rede e Redução: Tipos e Regras de Interação

A rede é representada pelos três símbolos de Lafont [3]: *gamma* (CON, aridade 2), *delta* (DUP, aridade 2) e *epsilon* (ERA, aridade 0). Cada agente possui um identificador único (*AgentId*, *u32* monotonicamente crescente) e portas indexadas por *PortId* ($u8 \in \{0, 1, 2\}$), onde 0 é a

porta principal e 1–2 são auxiliares. As conexões são armazenadas em um *port array* plano de tamanho $3 \times |A|$, onde o *slot id* $\times 3 + \text{port_id}$ contém a referência à porta conectada, o que dá acesso em $O(1)$. O tipo *PortRef* distingue dois casos: *AgentPort*(*AgentId*, *PortId*), para conexões entre agentes, e *FreePort*(*bid*), usado tanto para portas livres da interface original de Lafont [4] quanto como sentinela de fronteira introduzido pelo particionamento (Seção 4.4).

A detecção de *pares ativos* é incremental: a fila de *redexes* (*VecDeque*<(*AgentId*, *AgentId*)>) é mantida pela operação *connect*(*a*, *b*), que ao ligar duas portas principais insere o par automaticamente na fila e, assim, elimina a necessidade de varredura $O(N)$ por passo.

As seis regras de interação implementam fielmente a topologia definida por Lafont [3]:

Tabela 3: As seis regras de interação dos CI

Par Ativo	Padrão	Resultado	Balanco
CON–CON	Aniquilação	Reconexão cruzada das auxiliares	–2 ag.
DUP–DUP	Aniquilação	Reconexão paralela das auxiliares	–2 ag.
ERA–ERA	<i>Void</i>	Eliminação de ambos	–2 ag.
CON–DUP	Comutação	4 novos agentes (2 CON + 2 DUP)	+2 ag.
CON–ERA	Apagamento	2 novos ERA nas auxiliares do binário	0 ag.
DUP–ERA	Apagamento	2 novos ERA nas auxiliares do binário	0 ag.

O despacho usa uma tabela estática 3×3 indexada pelos símbolos do par, com complexidade $O(1)$. O laço *reduce_all* consome a fila de *redexes* até que esteja vazia (forma normal). Antes de aplicar qualquer regra, o motor de redução verifica a validade do par desenfileirado pela invariante I4 (*Redex Queue Validity*): ambos os agentes devem existir e suas portas principais devem estar mutuamente conectadas. Pares obsoletos, cujos agentes foram consumidos por interações anteriores, são descartados silenciosamente, sem aplicação de regras. Um contador de interações (*u64*) acumula o total de regras aplicadas. A invariante T7 garante que esse total é determinístico, independente da ordem de redução.

4.4 Particionamento e Merge de Sub-redes

O particionamento é definido por uma função de alocação $\sigma : A \rightarrow \{0, 1, \dots, n-1\}$ que atribui cada agente a exatamente um *worker*, induzindo uma partição matemática do conjunto de agentes. A implementação utiliza alocação *round-robin* ($\sigma(i) = i \bmod n$), estratégia que garante distribuição uniforme, embora não otimize a minimização de *wires* de fronteira. Os *wires* são classificados em três categorias: **internos** (ambas as extremidades na mesma partição), **de interface** (envolvendo *FreePort* pré-existente) e **de fronteira** (*border wires*, com extremidades em partições diferentes). Para cada *wire* de fronteira, a operação *split* substitui a conexão original por um par de sentinelas *FreePort*(*bid*) com identificador de borda único, registrados em um *border map* (*HashMap*<*u32*, (*PortRef*, *PortRef*)>).

Três condições de corretude garantem que o particionamento preserva a estrutura da rede. A condição **C1** (cobertura completa) exige que todo agente vivo pertença a exatamente uma partição; a **C2** (cobertura de *wires*), que todo *wire* seja classificado sem ambiguidade; a **C3** (bijetividade de *FreePort*), que cada *bid* apareça como *FreePort* em exatamente duas partições distintas. Juntas, essas condições formalizam a invariante D1 (*Split/Merge Identity*): $\text{merge}(\text{split}(\text{net})) \cong \text{net}$, base operacional da premissa P2. Cada *worker* recebe também um intervalo contíguo e exclusivo de *AgentId*, garantindo a premissa P4 (consistência de identificadores) sem coordenação em tempo de execução.

A operação *merge* reconstrói a rede a partir das partições reduzidas: une os agentes, copia as conexões internas e restaura as conexões de fronteira consultando o *border map*. A restauração invoca *connect(a, b)*, que detecta automaticamente novos pares ativos quando ambas as portas são principais, inclusive *redexes* de fronteira emergentes (que não existiam na rede original mas surgiram após redução local).

4.5 Protocolo de Coordenação e Computação em Grade

O ciclo de redução distribuída segue o modelo BSP em seis fases: (1) **particionamento**, em que o coordenador aplica *split* à rede global; (2) **despacho**, em que cada partição é enviada ao *worker* correspondente via TCP; (3) **redução local**, em que cada *worker* executa *reduce_all* sobre sua partição; (4) **coleta**, em que os *workers* retornam as partições reduzidas; (5) **merge e resolução de fronteira**, em que o coordenador recombina as partições e resolve *redexes* de fronteira via *reduce_all*; (6) **verificação de terminação**: se a fila de *redexes* está vazia, a rede atingiu a forma normal; caso contrário, inicia-se nova rodada.

A fase 5 trata um caso sutil, o dos *border redexes* emergentes. Considere-se a regra CON-DUP (comutação). Ao ser aplicada localmente, ela consome dois agentes e cria quatro novos. Se um dos agentes consumidos possuía uma porta conectada a *FreePort(bid)*, o novo agente herda essa conexão. Após o *merge*, quando o sentinela é resolvido para o agente correspondente na outra partição, pode formar-se um novo par ativo que não existia na rede original. A detecção ocorre na própria operação *connect(a, b)*. Ao reconectar as duas portas de um *bid*, ela verifica se ambas são portas principais e, em caso afirmativo, enfileira o par como *redex*. O mecanismo é iterativo: o *reduce_all* do coordenador consome todos os *border redexes* emergentes na mesma fase, sem necessidade de nova rodada BSP.

O protocolo de comunicação utiliza conexões TCP persistentes entre coordenador e *workers*, evitando o *overhead* de *handshake* por rodada. O *framing* emprega um cabeçalho de 8 bytes (4 bytes para comprimento do *payload* em *little-endian* e 4 bytes para *checksum* CRC32, *Cyclic Redundancy Check* de 32 bits), seguido do *payload* serializado via *serde* e *bincode* (bibliotecas de serialização binária do ecossistema Rust). Quatro tipos de mensagem compõem o catálogo: *AssignPartition* e *Shutdown* (coordenador para *worker*), *PartitionResult* e *Error* (*worker* para coordenador).

A terminação do protocolo (premissa P5) decorre da finitude das redes terminantes. Cada rodada consome ao menos uma interação do orçamento total determinístico (invariante T7), portanto o laço converge. A barreira BSP garante que o coordenador só executa o *merge* após receber todas as partições, preservando as condições C1–C3 entre rodadas.

4.6 Evoluções da Arquitetura na v2

A versão inicial do Relativist (v1, congelada em v0.10.0–bench) estabeleceu a base funcional descrita acima. A partir dela, seis evoluções arquiteturais foram especificadas e implementadas para sustentar empiricamente as premissas P2–P5 em redes maiores e em condições mais próximas de uma grade real. Todas seguiram o mesmo ciclo *Spec-Driven*: especificação formal → revisão adversarial → implementação TDD → *review* unificado → QA adversarial.

Abstração de transporte. O acoplamento direto entre coordenador e o *runtime* assíncrono foi substituído por uma interface que isola o protocolo lógico do meio físico, de modo que o transporte pode ser trocado sem reabrir as obrigações de prova da camada lógica.

Formato de transmissão v2. A serialização *bincode* permanece como padrão, complementada por um caminho *zero-copy* opcional (*rkyv*); a versão do protocolo é negociada no *handshake*.

Protocolo delta. Em vez de retransmitir cada partição reduzida na íntegra, o *worker* envia apenas a diferença em relação ao estado conhecido pelo coordenador (agentes consumidos, agentes criados, novos *wires*). A relevância acadêmica é direta, pois a viabilidade da computação em grade depende de o custo de comunicação não cancelar o ganho de paralelismo. Sem o protocolo *delta*, cada rodada BSP enviaria $O(|A|)$ bytes por *worker*; com *delta*, o tráfego escala com o tamanho da fronteira ativa em vez do tamanho total da rede. Esta evolução é o núcleo empírico verificado na Seção 5.

Geração streaming. O coordenador deixa de materializar a rede completa antes de particionar. A entrada é processada em *chunks*, e os *workers* solicitam trabalho conforme o consumo, o que remove a RAM do coordenador como gargalo arquitetural.

Gerência de arena. A arena de alocação ganhou reciclagem de identificadores em pilha e uma representação esparsa alternativa, mantendo o uso de memória limitado sob *streaming* de longa duração. A escolha entre as representações usa uma métrica de eficiência definida como a razão entre a memória útil e o intervalo de identificadores alocado.

Grade elástica. O coordenador foi promovido também a *worker*, e novos *workers* podem entrar em rodadas posteriores. Essa elasticidade é corolário operacional da confluência forte. Como o resultado independe de quem reduz cada par ativo, adicionar capacidade no meio da execução não invalida a corretude.

No conjunto, essas seis evoluções não alteram a tese central, pois as premissas P1–P6 permanecem, mas criam a infraestrutura empírica que permite, na Seção 5, separar o custo da redução distribuída em duas parcelas: o custo do modelo (particionamento e *merge*) e o custo do transporte (rede, *framing* e serialização).

4.7 Design Experimental

A suíte de *benchmarks* organiza-se em três perfis de *overhead*, derivados da distinção entre cargas *batch* e *streaming* de Mackie e Sato [11] (Seção 3):

Tabela 4: Perfis de *overhead* da suíte de *benchmarks*. A coluna *Border* indica *border redexes* emergentes por rodada.

Perfil	Descrição	Benchmarks	Rod.	Border
A	Emb. paralelo	EP-ERA, EP-CON, EP-DUP	1	0
B	Expansão + colapso	CON-DUP Exp., MixedNet, ChurchAdd, ChurchMul	Var.	Emerg.
C	Dep. sequencial	DualTree	d (prof.)	~50%/nív.

O Perfil A reúne os cenários mais favoráveis. Todos os *redexes* são independentes e se resolvem numa única rodada, o que torna o cenário ideal para distribuir. O Perfil B inclui a regra CON-DUP, a única que faz a rede crescer, e por isso gera *border redexes* novos a cada rodada; é o perfil mais próximo de uma aplicação real, em que a rede primeiro cresce e depois colapsa. Nele entram os *benchmarks* ChurchAdd e ChurchMul, que exercitam a aritmética de numerais de Church usada como primitivo pelos codificadores de cargas estruturadas. A evidência empírica de que a universalidade de Lafont [3] vale na prática, dentro do escopo deste trabalho, é a demonstração do método de Horner (Seção 5.7). Ali, a avaliação de polinômios é escrita inteiramente com as seis regras de interação, sem mecanismos auxiliares. O Perfil C é o pior caso, com forte dependência sequencial entre os níveis da árvore. O *TreeSum* entra como caso *data-bound* de *encoding* pragmático (soma pré-computada, pares ERA-ERA como unidades de trabalho), que valida a mecânica de distribuição sem exigir toda a expressividade formal do modelo.

Cada *benchmark* é executado com 1, 2, 4 e 8 *workers*, com variação logarítmica do tamanho da rede de entrada. A propriedade fundamental verificada em **cada ponto de dado** é:

$$\text{reduce_all}(\text{net}) \cong \text{run_grid}(\text{net}, n) \quad (1)$$

em que $\cong$ denota isomorfismo de grafos, isto é, igualdade estrutural a menos da renomeação dos identificadores. A verificação canonicaliza os identificadores de forma determinística e compara as estruturas. O critério de sucesso é a ausência de falhas de corretude em toda a suíte. Além da corretude, registram-se *speedup* (tempo sequencial dividido pelo distribuído), MIPS (milhões de interações por segundo), *overhead ratio* ($T_{\text{overhead}}/T_{\text{total}}$), *border ratio* (redes de fronteira sobre o total), eficiência (*speedup* sobre w) e o ponto de *break-even*, o menor tamanho a partir do qual distribuir é vantajoso. O relógio é monotônico (`std::time::Instant`); cada ponto de dado é a mediana de 10 repetições após 2 rodadas de *warmup*, acompanhado de intervalos de confiança de 95% para os tempos.

O design mede a equivalência entre a redução sequencial e a distribuída e decompõe o custo da distribuição, mas

não inclui comparação direta de desempenho com outros sistemas. A omissão é deliberada. O modelo aqui avaliado é um paradigma de redução distribuída ainda inicial, enquanto as plataformas de computação em grade ou de avaliação paralela com que se poderia confrontá-lo acumulam uma a duas décadas de otimização de engenharia. Uma comparação de tempo absoluto entre um protótipo de referência e essas plataformas maduras mediria sobretudo a diferença de maturidade de implementação, não a viabilidade do modelo. Por isso a avaliação se concentra em duas perguntas que independem dessa assimetria: a redução distribuída preserva o resultado da sequencial, e como o custo da distribuição se decompõe e escala com o tamanho da rede. O posicionamento qualitativo frente à literatura é retomado na Seção 5.

5 RESULTADOS E DISCUSSÕES

5.1 Ambiente Experimental

Os experimentos foram executados em um Lenovo ThinkPad T14 Gen 4 (Intel Core i7-1365U, 2 P-cores a 5,2GHz + 8 E-cores a 3,9GHz, 32GB DDR5-5200, Windows 11 Pro). Três ressalvas de hardware são documentadas para transparência: (1) o processador U-series sustenta PL1=15W indefinidamente, com turbo PL2=55W esgotado em segundos, de modo que *benchmarks* longos operam no regime *throttled*; (2) o agendador híbrido P/E do Windows distribui *threads* entre núcleos de desempenho e eficiência, introduzindo variância entre repetições (4 pontos de dado com CV entre 0,155 e 0,193); (3) na Etapa 2, o Docker Desktop (WSL2) aloca 12 vCPUs sobre 10 núcleos físicos. Estas limitações são aceitáveis porque a Etapa 3 em LAN comparará o *mesmo hardware* contra máquinas físicas em rede, isolando o custo de rede por subtração.

A campanha *baseline* utilizou o binário único v0.10.0-bench (rustc 1.94.1, perfil *release*), sob o protocolo de medição da Seção 4.7. A variância foi analisada por CV *triage* (limiar CV $\leq 0,15$). Dos 420 pontos de sumário, 63 foram sinalizados (58 por ruído de temporizador sub-milissegundo, 4 por agendamento P/E, 1 por ruído Docker) e todos classificados como *keep*, sem nenhuma exclusão nem re-execução.

O sistema é escrito em Rust (edição 2021, perfil *release*), compilado com o toolchain rustc 1.94.1; as versões exatas de todas as dependências, incluindo as transitivas, estão fixadas no Cargo.lock versionado, e a Tabela 13 (Seção 5.10) consolida a proveniência para reprodução. A concorrência e o transporte usam tokio 1.x, runtime assíncrono multi-thread que serve o protocolo TCP, e rayon 1.x, que fornece o paralelismo local empregado na comparação justa entre redução sequencial e distribuída. A serialização de estado combina serde 1.x com bincode 2.x para o formato de fio; crc32fast verifica a integridade de cada quadro, e lz4-flex 0.11 comprime o quadro antes da transmissão. A medição e os testes apoiam-se em criterion 0.5, que reporta os tempos com intervalo de confiança de 95%, e em proptest 1.x, que gera os testes baseados em propriedade da suíte de corretude.

Além da campanha *baseline* (3800 + 400 repetições), uma **campanha de stress** estendeu os tamanhos de rede para escala de produção: ep_annihilation_con até 50 milhões

de agentes e `dual_tree` até profundidade 25 (~67 milhões de agentes). O mesmo binário, hardware e metodologia foram utilizados, com 5 repetições por configuração (custo por repetição de 10–80 s em escala *stress*). A campanha *stress* durou ~67 min (Etapa 1) e ~106 min (Etapa 2), produzindo 290 execuções adicionais. No total, **4490 execuções** foram realizadas entre as duas campanhas.

5.2 Resultados da Etapa 1 — Redução Local (In-Process)

A Etapa 1 exercitou 12 *benchmarks* em modo *lenient* e 2 em modo *strict*, variando os *workers* em {sequencial, 1, 2, 4, 8}, ao todo 3800 repetições em 11 min 39 s. **Nenhuma falha de corretude ocorreu**: em cada ponto de dado, a propriedade fundamental (Equação 1) foi confirmada por isomorfismo de grafos.

A Tabela 5 resume os tempos medianos para os *benchmarks* representativos de cada perfil.

Tabela 5: Etapa 1: tempos medianos de *wall clock* (segundos) por configuração de *workers*. Todos os pontos com corretude verificada.

Benchmark	Seq. (s)	w=1	w=2	w=4	w=8	
ep_con 100k	0,013	0,014	0,034	0,036	0,044	
dual_tree 14	0,001	0,001	0,005	0,005	0,006	
condup_exp 5k	0,001	0,001	0,001	0,002	0,002	
cascade_cross 1k	0,000	—	—	—	0,000	

O principal achado da Etapa 1 é que, em modo local, não existe *break-even*. Dos 272 *speedups* medidos com *grid* local, 44,9 % ficaram abaixo de 0,1, ou seja, o *grid* chegou a ser mais de 10× mais lento que o sequencial, e 78,4 % não passaram de 0,5. Só 4 configurações superaram *speedup*=1,0, todas com um único *worker*: o ganho veio do *cache* de um *subnet* compacto, não de paralelismo real. **Com dois ou mais workers, nenhuma configuração superou o sequencial**. O motivo é duplo. Quando a comunicação é instantânea, o custo $O(|A|)$ de particionar e recombinar domina o tempo total; e, com apenas 2 P-cores físicos no i7-1365U, qualquer configuração de três ou mais *workers* disputa os mesmos núcleos.

Validação em escala de produção. Para verificar se os achados da *baseline* se mantêm em redes maiores, a campanha de *stress* levou `ep_annihilation_con` a 50 M de agentes (correspondentes a 100 M de nós de entrada) e `dual_tree` à profundidade 25 (~67 M de agentes), somando 150 repetições em cerca de 67 min. Novamente, **nenhuma falha de corretude ocorreu** em todos os pontos, inclusive na rede de 50 M de agentes.

A Tabela 6 mostra os tempos medianos e a razão de *slowdown* com 2 *workers*.

O resultado mais significativo é a **invariância de escala do overhead** relativo. Em `ep_annihilation_con`, a razão $w=2/\text{Seq}$ varia pouco: 2,78× em 10 M, 2,80× em 20 M e 2,66× em 50 M, menos de 5 % de variação. Em `dual_tree`, o padrão se repete, com 4,54× em $d=23$ e 4,16× em $d=25$. A conclusão é que o custo de distribuir é **proporcional ao tamanho da rede**, não um custo fixo de *startup* que se dilui à medida que a rede cresce. A causa é que cada rodada BSP

Tabela 6: Campanha *stress* (Etapa 1): tempos medianos de *wall clock* (segundos) em escala de produção. Valores de `wall_clock_median` dos CSVs congelados. 5 repetições/config, 2 *warmup*.

Benchmark	Seq.	w=1	w=2	w=4	w=8	w=2/Seq
ep_con 10M	3,47	3,54	9,64	10,68	13,27	2,78×
ep_con 20M	7,37	7,62	20,67	23,17	28,67	2,80×
ep_con 50M	20,45	20,76	54,49	60,47	76,01	2,66×
dual_tree 23	1,55	1,57	7,04	7,12	7,30	4,54×
dual_tree 24	3,52	3,50	14,75	14,93	15,42	4,19×
dual_tree 25	7,44	7,49	31,00	31,16	32,14	4,16×

reconstrói e reduz a rede inteira no coordenador, operação que escala linearmente com $|A|$.

Esse custo proporcional ao tamanho da rede é coerente com a lição de Mackie [6], para quem o particionamento domina o desempenho da redução distribuída de *interaction nets*. O contraste com o HVM2 [5] delimita o que se reivindica aqui. Aquele avaliador obteve *speedup* quase linear, mas em memória compartilhada, em que não há transporte nem recombinação. Este trabalho não reivindica ganho de paralelismo em *localhost*, onde o transporte e o *merge* no coordenador superam o trabalho útil de redução. A vantagem do modelo distribuído fica restrita ao regime de rede com banda limitada, examinado na Seção 5.5.

Validação do modo strict BSP. O modo *strict* confirma as previsões teóricas para o número de rodadas com casamento exato:

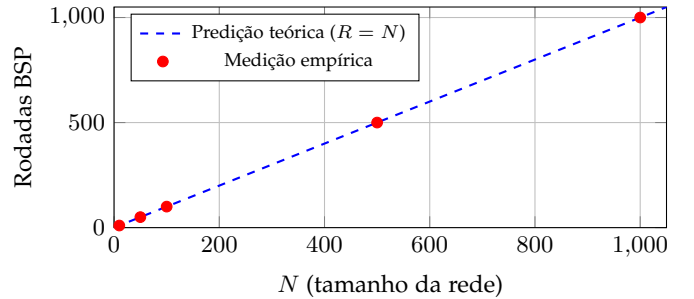


Fig. 4: Validação do modo *strict* BSP: rodadas empíricas do `cascade_cross(N)` coincidem exatamente com a predição teórica $R = N$ para $w \geq 2$ *workers*. Diagrama elaborado pelo autor em TikZ/pgfplots com auxílio de IA.

O `cascade_cross(N)` termina em exatamente N rodadas e o `dual_tree(d)` em d rodadas (com *workers* ≥ 2), confirmando que o protocolo BSP preserva a semântica da redução distribuída e que a detecção de *border redexes* emergentes funciona corretamente entre rodadas.

5.3 Resultados da Etapa 2 — TCP Localhost sob a v2

A versão v2 do Relativist (o pré-lançamento v0.20.0-pre da Seção 4.6) foi reavaliada sob protocolo TCP *localhost* em contêineres Docker, exercitando de uma só vez a geração *streaming*, o protocolo *delta* de transferência e a gerência de arena de memória. A campanha reúne 32 configurações distribuídas, cada uma com 10 repetições, mais 8 *baselines* sequenciais nativos, cada um com 10 repetições, num total

de 400 pontos de dado (320 distribuídos e 80 sequenciais). As 32 configurações confirmaram a corretude estrutural em 100% das repetições. Somados aos da v1 (4490 execuções, das quais 4465 completaram a verificação; as 25 restantes abortaram por um limite de *frame* TCP, conforme as Limitações), esses números totalizam cerca de 4800 reduções distribuídas verificadas contra a redução sequencial, sem uma única divergência de corretude. A Tabela 7 resume os tempos medianos.

Tabela 7: Etapa 2 (v2): tempos medianos de *wall clock* (segundos) via TCP *localhost* em Docker, campanha v2_post_d012_baseline_2026-05-05. Todos os pontos com *all_correct* = true.

Benchmark	Seq. (s)	w=1	w=2	w=4	w=8
ep_con 500k	0,368	0,460	0,575	0,680	1,135
ep_con 1M	0,576	0,935	1,172	1,339	2,043
ep_con 5M	3,185	5,815	7,403	7,959	11,805
dual_tree 18	0,288	0,296	0,448	0,518	0,749
dual_tree 20	0,483	0,830	1,438	1,605	1,942
dual_tree 22	1,463	3,674	6,335	7,227	8,153

Comparação v2 vs. v1. As mesmas configurações foram reavaliadas no binário v1 congelado (v0.10.0-bench), permitindo uma razão direta v2/v1 por família de *benchmark*:

Tabela 8: Razão v2/v1 de *wall clock* mediano em TCP *localhost*. [†] *condup_expansion* opera em escala muito reduzida (1 k–5 k agentes) e tempos absolutos abaixo de 60 ms; o *ratio* reflete custo fixo de *handshake* e *framing* amortizado sobre carga ínfima e não generaliza para regimes de produção.

Família	N slots	v2/v1 mediana	Faixa
ep_annihilation_con	12	1,63×	1,43 – 2,06
dual_tree	12	1,77×	1,26 – 3,04
condup_expansion [†]	8	12,8×	7,04 – 25,4

A interpretação deste resultado depende do regime de comunicação. A v2 custa $\sim 1,6\text{--}1,8\times$ mais em *localhost* para entregar a mesma forma normal; a Seção 5.5 mostra por que essa sobrecarga se converte em vantagem em rede real.

Eficiência do protocolo *delta*. Apesar do custo adicional em *wall clock*, a v2 transmite menos *bytes* pela rede do que a v1 envia em direção única, pois o protocolo *delta* transporta diferenças, não estados completos. Para a configuração *ep_annihilation_con* 500 k com $w=1$, na primeira rodada:

Tabela 9: *Bytes* trafegados por rodada no caminho coordenador–*worker*. Apesar de a v2 introduzir tráfego bidirecional (envio do *delta* de retorno), o total é 30% inferior ao da v1 unidirecional. Fonte: *rounds.csv*, *ep_annihilation_con* 500 k $w=1$, rodada 0.

Versão	Coord. → W	W → Coord.	Total
v1	41,00 MB	< 1 KB	41,00 MB
v2 (<i>delta</i>)	23,80 MB	4,89 MB	28,69 MB
Variação	–42 %	—	–30 %

A assimetria do tráfego v1 (41 MB ida, ~ 153 B volta) decorre de o coordenador enviar a partição completa e receber

apenas um identificador de resultado; a v2, por construção, envia também um *delta* de retorno. A comparação justa é sobre o total trafegado, 30% menor na v2.

Campanhas v1 históricas preservadas. A Tabela 5 (Etapa 1 *in-process*) e a Tabela 6 (*stress* v1) permanecem válidas como referência da v1, pois as evoluções da Seção 4.6 introduziram novos caminhos de código que não existiam na v1, e portanto os *ratios* de escala observados na v1 não foram retroativamente alterados.

5.4 Decomposição do Custo por Rodada

A instrumentação introduzida nessa fase da v2 expôs, pela primeira vez, os tempos por componente em cada rodada BSP, a saber, particionamento, computação local, *merge* e rede, todos coletados por *worker* e reportados ao coordenador via *delta*. A Tabela 10 apresenta a média por componente sobre 320 rodadas distribuídas TCP *localhost* da campanha canônica.

Tabela 10: Decomposição média por rodada em TCP *localhost* (v2, 320 rodadas). A componente de rede domina mesmo em *loopback*, porque a v2 negocia estado bidirecional a cada rodada. O resíduo concentra *framing*, orquestração FSM e operações da arena.

Componente	Tempo médio (s)	Fração
Particionamento (<i>split</i>)	0,0009	0,04 %
Computação local (<i>reduce_all</i>)	0,192	8,0 %
<i>Merge</i> no coordenador	0,145	6,0 %
Rede (<i>framing</i> + TCP)	1,139	47,3 %
Resíduo (orquestração + alocação)	0,933	38,7 %
Total medido	2,409	100 %

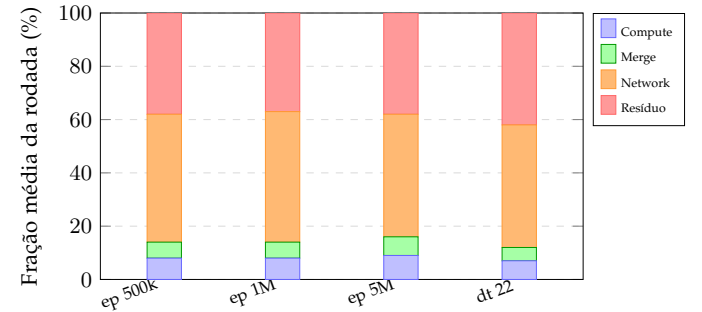


Fig. 5: Decomposição percentual média do tempo por rodada em TCP *localhost* (v2). A componente de particionamento ($< 0,1\%$) é omitida por inviabilidade visual. *Network* domina sistematicamente ($\sim 46\text{--}49\%$), seguida pelo resíduo de orquestração; *compute* e *merge* somam menos de 15%. Diagrama elaborado pelo autor em TikZ/pgfplots a partir de *rounds.csv*, com auxílio de IA.

A estimativa indireta da v1 (razão Etapa 2 / Etapa 1) dá lugar a uma medição direta. Em TCP *localhost*, cerca de 47% de cada rodada vão para a rede, sem nenhuma rede física no caminho. Esse tempo corresponde ao *kernel* TCP, ao *framing* *bincode* e à serialização do *delta*; numa LAN física, parte desse trabalho ocorreria em paralelo com a transmissão real. O resíduo ($\sim 38\%$) reúne a máquina de estados da geração *streaming*, as operações de arena e o *framing* de cabeçalho.

Compute e *merge*, somados, ficam abaixo de 15% do tempo. Reduzir a rede, o trabalho próprio do modelo de Lafont, não é o gargalo.

5.5 Análise de Break-Even e Projeção para LAN

A v1 deste trabalho derivou empiricamente, a partir da Etapa 1 e da Etapa 2, uma razão $c_o/c_r \approx 2,2$ entre o custo *overhead* de distribuição e o custo de redução local. A v2 refina esta análise com dados por componente.

Em relação à v1, a v2 paga aproximadamente **100 ms adicionais de CPU por rodada** (camada de transporte e *framing*) e **economiza ~12 MB por rodada no fio** (Tabela 9). O *break-even* de banda é o ponto a partir do qual a economia de *bytes* supera o custo de CPU. Ele decorre em dois passos. O piso ingênuo divide a economia de *bytes* pelo custo bruto de CPU; descontando a parcela de *framing* de cabeçalho (~30% do *payload*), que não disputa com a transmissão útil, o tempo de CPU efetivamente atribuível à troca cai para ~77 ms e eleva o ponto de equilíbrio:

$$B_{eq}^{bruto} = \frac{\Delta \text{ bytes}}{\Delta \text{ tempo de CPU}} = \frac{12 \text{ MB}}{0,100 \text{ s}} = 120 \text{ MB/s},$$

$$B_{eq} = \frac{12 \text{ MB}}{0,077 \text{ s}} \approx 156 \text{ MB/s} (\approx 1,2 \text{ Gb/s}).$$

A v2 tende a superar a v1 em redes mais lentas que 156 MB/s, faixa que abrange a banda típica de LAN doméstica e de WAN. Em *localhost*, onde a banda é, na prática, ilimitada, a troca não se justifica e a v2 perde, como mostram as Tabelas 7 e 8. Logo, *localhost* não é o regime adequado para avaliar essa otimização, pois o ganho do protocolo *delta* depende de um tempo de transmissão que, ali, não existe.

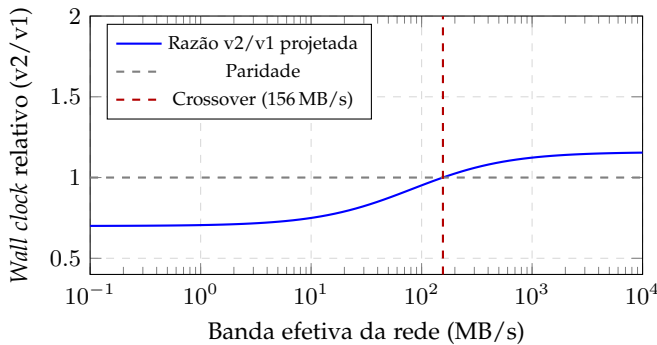


Fig. 6: Projeção do *ratio* v2/v1 em função da banda efetiva da rede, derivada da decomposição da Tabela 10 e do volume de *bytes* da Tabela 9. À direita do *crossover* (banda alta, *localhost*), a v2 perde; à esquerda (LAN típica, ~100 MB/s; WAN, ~10 MB/s), a v2 ganha. A medição empírica em LAN física (Etapa 3) é objeto da Seção 6.3. Diagrama elaborado pelo autor em TikZ/pgfplots, com auxílio de IA.

Esse *crossover* é a assinatura empírica que o argumento teórico do trabalho exigia. É o ponto de banda em que a camada distribuída deixa de ser apenas sobrecarga, agora decomposto em componentes mensuráveis. Ele é derivado, não medido. A confirmação direta depende de execução em LAN física, tratada na Seção 6.3.

5.6 Verificação Estendida sob Varredura Geométrica de N

As campanhas da Etapa 1 e da Etapa 2 reportadas até aqui amostram, para cada *benchmark*, seis pontos de tamanho até aproximadamente 50 M agentes. Para observar a propriedade central ao longo de uma *curva*, e não em pontos isolados, uma campanha auxiliar varreu a dimensão N de forma mais fina e mais ampla, sobre uma única carga representativa, a aniquilação CON-CON (*ep_annihilation*). Os tamanhos foram espaçados geometricamente por um fator $\sqrt{10}$, de 10^4 a 10^7 , em sete pontos que cobrem quatro ordens de grandeza. O mesmo binário executou em dois ambientes, redução local no mesmo processo e *localhost* via TCP em contêineres, cada um com quatro graus de paralelismo ($W \in \{1, 2, 4, 8\}$) e a redução sequencial como linha de base.

O dado central é que **as 245 execuções do braço local e as 240 do braço TCP convergiram, todas, à mesma forma normal da redução sequencial**, sem nenhuma divergência estrutural ao longo de quatro ordens de grandeza de N e de quatro configurações de paralelismo. A Figura 7 mostra o tempo de parede em função de N no braço local (eixos log-log) e a Tabela 11 resume o braço TCP. Nos dois ambientes, as curvas sequencial e distribuída sobem com N mantendo separação constante. A sobrecarga relativa da camada distribuída não aumenta com N ; é multiplicativa, em acordo com a análise da Seção 5.5.

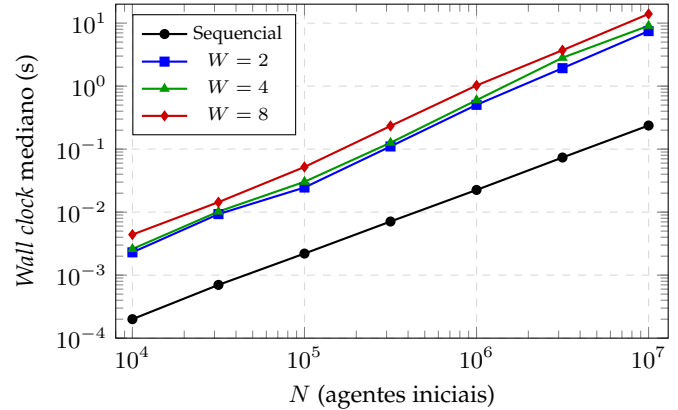


Fig. 7: Tempo de parede mediano em função de N na varredura geométrica (*ep_annihilation*, braço *in-process*). Eixos em escala logarítmica. As curvas distribuídas ($W \in \{2, 4, 8\}$) acompanham a curva sequencial mantendo separação multiplicativa estável ao longo de quatro ordens de grandeza; a correte estrutural foi verificada em 100% dos 245 pontos. Diagrama elaborado pelo autor em TikZ/pgfplots, com auxílio de IA.

A campanha tem três limitações. A primeira é a **carga única**. Três cargas foram originalmente planejadas para esta varredura: aniquilação CON-CON, comutação CON-DUP e expansão DUP-DUP. Apenas a primeira foi executada nesta entrega; estender às outras duas consta da Seção 6.3. A segunda é o **teto em $N = 10^7$** . O corte decorre de uma proteção operacional contra alocações excessivas de memória; a extensão original visava 10^9 , que exigiria cerca de 68 GB de RAM por execução. A terceira é o **abort do**

Tabela 11: Varredura geométrica no braço TCP *localhost* (Docker): tempo de parede mediano (segundos) por configuração de paralelismo. Linhas intermediárias da progressão $\times \sqrt{10}$ omitidas por concisão; os sete pontos completos estão na campanha congelada. No ponto $N = 10^7$, o braço Docker abortou após três tentativas consecutivas (provavelmente exaustão de memória no contêiner), limitação operacional do ambiente e não do protocolo; a corretude foi verificada em 100 % dos 240 pontos que completaram.

N	Seq.	$W=1$	$W=2$	$W=4$	$W=8$	
10^4	0,0002	0,0003	0,0033	0,0035	0,0051	
10^5	0,0023	0,0028	0,0563	0,0549	0,0548	
10^6	0,0216	0,0257	0,9307	0,9899	1,0958	
$3,16 \times 10^6$	0,0768	0,0914	3,7627	4,2484	4,7431	
10^7	—	abort (instabilidade do contêiner)				

braço TCP em $N = 10^7$, causado por instabilidade do contêiner sob carga máxima, não pela camada de protocolo. Um detalhe técnico adicional surgiu na configuração $N = 10^7$, $W = 8$ do braço *in-process*. O particionamento *round-robin* por intervalos de `AgentId` deixou alguns *workers* com sub-redes vazias. Isso acionou o caminho de *fallback* “*empty-Net*”, sem prejuízo à corretude, pois todas as repetições convergiram à mesma forma normal, mas mostrou que a partição padrão não é ideal para cargas ultrarregulares nessa escala (Seção 5.5 e Limitações, adiante). Somadas, essas 485 execuções reforçam a evidência das campanhas principais e fundamentam a discussão das premissas a seguir.

5.7 Demonstração com Carga Estruturada: Avaliação de Polinômios

As campanhas anteriores exercitam cargas **sintéticas**, a saber, aniquilação CON-CON, comutação CON-DUP e árvores duais, todas projetadas para isolar regras específicas de interação. Elas respondem à pergunta da corretude estrutural, mas deixam uma questão aberta: a propriedade central também vale sob uma carga que codifica um **algoritmo real**, cujo resultado pode ser conferido contra um oráculo aritmético externo? Para responder, implementou-se uma demonstração baseada no **método de Horner** para avaliar polinômios. O método usa a forma encadeada $((c_n \cdot x + c_{n-1}) \cdot x + \dots) \cdot x + c_0$, que resolve um polinômio de grau n com n multiplicações e n adições. A rede codifica diretamente essa sequência de operações sobre numerais de Church; decodificada, sua forma normal deve coincidir com o valor aritmético esperado.

A bateria reúne dez polinômios distintos (Tabela 12), com grau de zero a dois, do caso trivial (a constante 42) a casos com valor avaliado na casa dos milhões (o polinômio $10\,000 + 1025x$ em $x = 10\,000$ resulta em $10\,260\,000$). Cada polinômio executou em quatro graus de paralelismo ($W \in \{1, 2, 4, 8\}$) e dois ambientes, o *in-process*, no mesmo processo, e o *localhost* via TCP em contêineres, totalizando **80 execuções** (10 polinômios $\times$ 4 *workers* $\times$ 2 ambientes). Cada execução fornece dois testes de corretude independentes: o **valor numérico** lido da forma normal, conferido contra o oráculo aritmético, e a **forma normal estrutural** do braço distribuído, comparada por isomorfismo de grafos à do

braço sequencial *in-process* (essa segunda comparação não se aplica ao próprio *in-process*, que serve de referência).

Tabela 12: Bateria de demonstrações Horner: dez polinômios cobrindo graus 0 a 2 e magnitudes variadas de coeficientes e ponto de avaliação. Cada linha foi executada em quatro configurações de paralelismo ($W \in \{1, 2, 4, 8\}$) e dois ambientes (*in-process* e TCP em contêineres), totalizando 80 execuções. A coluna **Inter.** reporta o número de interações de redução elementares aplicadas para atingir a forma normal, uma medida do trabalho computacional da rede, independente do tempo de parede. O valor decodificado da forma normal coincidiu com o esperado em 100 % das execuções; nas 40 execuções do braço distribuído, a forma normal coincidiu estruturalmente com a do braço sequencial em 100 % dos casos.

#	Gr.	Polinômio	x	Esperado	Inter.
D1	0	42	99	42	0
D2	1	$1 + x$	5	6	11
D3	1	$100 + x$	50	150	11
D4	1	$42 + x$	10 000	10 042	11
D5	1	$1 + 1025x$	10 000	10 250 001	2059
D6	1	$10\,000 + 1025x$	10 000	10 260 000	2059
D7	2	$1 + x + x^2$	2	7	26
D8	2	$1 + x^2$	3	10	28
D9	1	$3 + 5x$	4	23	19
D10	2	$10\,000 + 500x + x^2$	100	70 000	1220

Os dois critérios foram atendidos. Nas **80 execuções**, o valor decodificado da forma normal foi exatamente o esperado e, nas 40 do braço distribuído, a forma normal estrutural foi isomorfa à do braço sequencial *in-process*. Assim, a propriedade central deste trabalho, a equivalência entre redução sequencial e distribuída, vale também para uma carga com significado, confirmada por dois caminhos independentes: um oráculo numérico externo e o isomorfismo de grafos interno. O número de interações cresce com a magnitude e o grau do polinômio, de zero (a constante, que já está em forma normal) a 2059 (o caso de maior magnitude do braço linear). Isso indica que a rede *computa* o polinômio, e não apenas colapsa para uma constante embutida.

Cabe um esclarecimento de escopo. Em todas as 80 execuções, o tempo de parede ficou abaixo do que o relógio consegue medir (menos de 10 ms), e o máximo de interações (2059) é várias ordens de grandeza menor que o das campanhas principais. Por isso, **esta é uma demonstração qualitativa, não uma medição de desempenho**. Ela mostra que o codificador/decodificador distribuído funciona sobre uma carga estruturada e que a equivalência estrutural se mantém sob distribuição, mas nada afirma sobre eficiência nesse regime. A demonstração opera dentro de um envelope definido. Cada coeficiente e o ponto de avaliação x são representados por numerais de Church, codificação unária cujo custo em agentes cresce com o valor. Por isso, o codificador limita cada escalar de entrada a 10 000; o decodificador, por sua vez, lê de volta o resultado para polinômios de grau até dois (no grau um, com coeficiente angular até 1 025; no grau dois, com coeficiente líder unitário). Dentro desse envelope, o valor avaliado decodificado não tem teto próprio: o caso $10\,000 + 1025x$ em $x = 10\,000$ é lido corretamente como 10 260 000. Ampliar o envelope do decodificador para graus

e coeficientes maiores é uma das frentes de trabalhos futuros (Seção 6.3).

5.8 Validação das Premissas P1–P6

As seis premissas que sustentam a hipótese do trabalho foram validadas pelos dados experimentais:

- **P1 (Confluência forte):** garantida por construção [4, Proposição 1] (Seção 2.1).
- **P2 (Corretude do particionamento):** identidade $\text{merge}(\text{split}(\text{net})) \cong \text{net}$ confirmada em 100 % dos 4465 pontos da v1 e dos 320 pontos distribuídos da v2 (mecanismo na Seção 4.4).
- **P3 (Compleitude de fronteira):** nenhum *redex* de fronteira perdido nas execuções multi-rodada (detecção descrita na Seção 4.4).
- **P4 (Consistência de identificadores):** garantida pela alocação de intervalos disjuntos (Seção 4.4).
- **P5 (Terminação do protocolo):** rodadas medidas coincidem com a predição teórica (Fig. 4).
- **P6 (Redes terminantes):** todos os *benchmarks* terminam por construção.

A propriedade fundamental (Equação 1) foi satisfeita em **100 % das 4465 execuções com corretude verificada** da v1 e em **100 % das 320 execuções distribuídas da v2**, ou seja, cerca de 4800 reduções verificadas contra a redução sequencial, sem divergência estrutural.

5.9 Limitações

- 1) **Ausência de validação em rede física.** Em TCP *localhost*, a banda efetiva está acima do *crossover* de 156 MB/s derivado na Seção 5.5, de modo que nem a v1 supera o sequencial nesse regime, nem a v2 supera a v1. A região onde a distribuição é vantajosa, isto é, LAN real a ~ 100 MB/s, é o objeto da Etapa 3 (Seção 6.3), cujo protocolo operacional acompanha o pré-lançamento v0.20.0-pre do Relativist.
- 2) **Particionamento *round-robin*.** A estratégia atual não otimiza a minimização de *wires* de fronteira (Seção 6.3).
- 3) **Coordenador único, topologia estrela.** A escalabilidade é limitada pela largura de banda do *merge* centralizado. A v2 já introduz, em modo de detecção, a admissão dinâmica de *workers* e o uso do próprio coordenador como *worker*. O caminho completo de reincorporação de um *worker* após saída é tratado na Seção 6.3.
- 4) **Hardware de *benchmark*.** Os tempos absolutos refletem um *ultrabook* com *throttling* térmico. Os *ratios* (*speedup*, eficiência) são mais confiáveis que os valores absolutos.
- 5) **Limite de *frame* TCP resolvido na v2.** A v1 limitou cada *frame* TCP a 1 GiB, abortando 25 execuções da campanha *stress*. A v2 resolveu essa limitação pela geração *streaming* (Seção 4.6).

5.10 Reprodutibilidade

Todos os artefatos experimentais deste trabalho são públicos e verificáveis. O binário usado nas campanhas da Etapa 1

e da Etapa 2 da v1 é a *tag* v0.10.0-bench do repositório público do Relativist; a evolução v2 é o pré-lançamento v0.20.0-pre do mesmo repositório. Três campanhas estão empacotadas e congeladas no repositório: a Etapa 1 *in-process*, a Etapa 2 em TCP *localhost* sob contêineres Docker e a varredura geométrica em *N*. Cada uma traz um manifesto que registra a procedência (versão do binário, ambiente, comandos exatos) e as somas de verificação criptográficas dos dados brutos. O roteiro para reproduzir a bateria em LAN física acompanha o mesmo pré-lançamento como anexo (Seção 6.3). A Tabela 13 consolida os itens mínimos para reproduzir as campanhas.

Tabela 13: Proveniência para reprodução. O *toolchain* foi registrado nos manifestos das campanhas congeladas; as dependências, incluindo as transitivas, estão fixadas no Cargo.lock versionado.

Item	Valor
<i>Toolchain</i>	rustc 1.94.1 (2026-03-25) / cargo 1.94.1; edição 2021, perfil <i>release</i>
Binários	v0.10.0-bench (v1), v0.20.0-pre (v2)
Dependências	versões exatas (com as transitivas) fixadas no Cargo.lock versionado
Sistema	Windows 11 Pro; Etapa 2 em contêineres Docker Desktop sobre WSL2
Dados brutos	três campanhas congeladas, cada uma com manifesto de proveniência e somas SHA-256

6 CONCLUSÃO E TRABALHOS FUTUROS

6.1 Síntese e Contribuições

A hipótese deste trabalho foi confirmada em cada ponto medido. Ela afirma que a confluência forte dos CI, somada a um protocolo que preserva a estrutura da rede (premissas P1–P6, Seção 4), leva a forma normal distribuída idêntica à da sequencial em redes terminantes. Nas 5350 reduções verificadas entre as campanhas principais, a varredura geométrica em *N* e a demonstração de Horner (a única conferida por oráculo numérico externo e isomorfismo de grafos), o resultado foi 100 % de formas normais idênticas (Seção 5). A consistência cobre quatro ordens de grandeza no tamanho da rede (10^4 a 10^7 agentes), até oito *workers* em paralelo, e vale para cargas sintéticas e estruturadas.

Dessa evidência, o trabalho extrai quatro contribuições. A **teórica** é identificar um conjunto coeso de premissas operacionais que a redução distribuída precisa cumprir para que a confluência forte de Lafont [3] se converta em determinismo de resultado em uma grade. São as P1 a P6 da Seção 4, cada uma associada a um invariante verificável nas especificações da implementação. A de **engenharia** é o próprio Relativist, construído sob *Spec-Driven Development* e publicado como referência operacional do modelo, em código aberto e reprodutível. A **empírica** é a bateria congelada de experimentos. Até o presente momento, esta é a primeira validação empírica dessa equivalência entre redução sequencial e distribuída de CI numa arquitetura de grade. A **metodológica**, registrada na Seção 6.2, mostra como os invariantes formais e os testes empíricos se complementam em períodos de refatoração agressiva.

Resta delimitar o que essa evidência prova e o que não prova. O que está afirmado é a **equivalência estrutural** entre a redução sequencial e a distribuída, a propriedade central da hipótese, confirmada por isomorfismo de grafos em todas as execuções concluídas. Essa confirmação não depende da topologia física da rede. Basta que o protocolo preserve a estrutura da rede de uma rodada para a outra; por isso, mesmo o resultado em *localhost* sustenta a tese. O que falta é a **caracterização de desempenho em rede real**. Como todos os experimentos executaram em *localhost* (processo único e contêineres TCP num só *host*), a análise de *break-even* (Seção 5.5) apenas projeta, sem medir em LAN, a partir de que banda a distribuição passa a ser vantajosa. Esse é o ponto de partida dos trabalhos futuros.

6.2 Invariantes formais como camada ortogonal de cobertura

Um achado metodológico emergiu, de forma não planejada, na transição entre as versões v1 e v2 do Relativist. Ao reavaliar a campanha TCP *localhost* sob a v2, observou-se regressão de tempo de parede em uma carga representativa (aniquilação CON-CON, cinco milhões de agentes, dois *workers*), de cerca de 14,2 s na v1 para perto de 22 s, um aumento de 83 %.² Uma bisseção em sete pontos do histórico apontou a causa. Numa refatoração da camada de partição, uma métrica de tamanho da arena passou a medir o intervalo de identificadores planejado, e não o espaço ocupado pelos agentes vivos, de modo que partições válidas eram roteadas para um caminho de *fallback* ineficiente. A correção do roteamento expôs **dois bugs latentes** no caminho otimizado, introduzidos pela mesma refatoração e ausentes na v1: um comprometia a transferência incremental de *deltas*; o outro permitia ao coordenador reutilizar identificadores já alocados a um *worker* remoto. O segundo foi capturado em tempo de execução pelo predicado automático que monitora a consistência da rede, ao colidir em uma carga adjacente; o primeiro, por leitura adversarial do contrato da carga útil *worker-coordenador*. A especificação foi emendada (métrica corrigida, dois *guards* nos pontos de criação de agente e de *merge*); cada *bug* foi corrigido com duas linhas de código, e a campanha pós-correção devolveu a razão a $1,11\times$ ante a v1, dentro do piso de ruído da medição.

A lição não é que a v2 seja, *a posteriori*, mais correta do que a v1. Os dois *bugs* nasceram na própria refatoração da v2. Também não se trata de afirmar que invariantes formais detectam o que testes empíricos não detectam. Testes baseados em propriedade, com cobertura de cargas ampla o bastante, provavelmente capturariam boa parte desses casos. O ponto defensável é mais modesto e **complementar**. Os predicados de invariante não dependem da carga e disparam no instante em que o estado intermediário viola a propriedade, sem exigir que o autor anteveja o espaço adversarial; os testes de propriedade cobrem o eixo *carga* $\times$ *saída*. Os dois eixos são ortogonais e, juntos, alcançam estados intermediários que as cargas testadas, sozinhas, não

revelariam. O registro de incidentes do projeto reúne sete episódios em seis semanas, todos com o mesmo padrão: os testes dos primitivos passavam, a integração falhava em silêncio, e o contrato de uma especificação ou de um invariante detectava o desvio. A janela é curta, não há grupo de controle e o processo misturou o trabalho do autor com ferramentas de análise automatizada; essas limitações ficam aqui declaradas. Ainda assim, o padrão sugere que, em fases de refatoração agressiva, manter os invariantes formais como camada separada dos testes empíricos encurta o tempo até a detecção. As emendas de especificação e o relatório pós-incidente estão versionados no pré-lançamento v0.20.0-pre do Relativist.

6.3 Trabalhos Futuros

O trabalho deixa quatro frentes naturais de continuação, agrupadas pela natureza da extensão que cada uma representa: experimental, empírica, de codificação e arquitetural. Cada uma se conecta diretamente a um limite declarado na Seção 6.1.

Frente experimental — validação em LAN física. O passo que fecha o ciclo proposto é executar a bateria distribuída em uma rede local, com pelo menos duas máquinas físicas heterogêneas. Trocar *localhost* por uma LAN substitui a latência de *loopback*, de poucos microssegundos, por latência de rede real, de centenas de microssegundos a milissegundos, o que desloca o ponto de *break-even* (Seção 5.5) para a faixa em que distribuir as cargas mais pesadas se torna vantajoso. A execução exige cuidado operacional e está detalhada no *release* v0.20.0-pre do Relativist: sincronizar relógios via NTP, abrir as regras de *firewall* TCP e comparar as serializações *bincode* e *rkyv* em modo *zero-copy*.

Frente empírica — generalização da varredura. A varredura geométrica da Seção 5.6 cobriu apenas uma das três cargas planejadas, a aniquilação CON-CON. As outras duas são a comutação CON-DUP e a expansão DUP-DUP. Estendê-la a elas e elevar o teto de N a 10^9 (em ambiente com 128 GB de RAM ou mais) revelaria em que escala cada classe de regra de interação passa a dominar o custo total, refinando a decomposição apresentada na Seção 5.

Frente de codificação — generalização do codificador polinomial. O codificador de Horner usado nas dez demonstrações da Seção 5.7 opera num envelope estreito: polinômios de grau até dois, coeficientes inteiros não-negativos e valores intermediários dentro do teto do codificador de numerais de Church ($\sim 10\,000$). Ampliá-lo exige três passos. O primeiro é liberar o grau. O módulo de leitura inversa de redes precisa reconhecer árvores de duplicação compostas, de *fan-out* alto. O segundo é aceitar coeficientes negativos e racionais, o que exige ir além dos numerais de Church, por exemplo com pares Church-codificados que representem inteiros relativos ou frações. O terceiro é elevar o teto de magnitude acima de 10^4 , o que exige representação numérica mais compacta que a dos numerais de Church unários, já que estes fazem o número de agentes crescer linearmente. Codificadores para outros domínios, como avaliação multipontos de polinômios, multiplicação de matrizes e recorrências, dariam origem a cargas distribuídas por construção, com paralelismo de dados embutido; seriam mais favoráveis ao paralelismo do que as cargas sintéticas

2. Os valores absolutos citados nesta subseção provêm de um *harness* de reprodutibilidade que executa o binário diretamente em ambiente nativo. Eles diferem dos valores reportados na Tabela 7 (Seção 5), obtidos em contêiner Docker com normalização de *warmup* e descarte da primeira repetição. A razão de $1,11\times$ entre v1 e v2 pós-correção, citada ao final da subseção, é estável entre os dois *harnesses*.

das campanhas principais ou a própria demonstração de Horner, que é sequencial encadeada por natureza.

Frente arquitetural — interfaces e robustez. Restam quatro continuações naturais da implementação. A primeira trata da admissão dinâmica de *workers*, entregue apenas em modo de detecção. Concluir o caminho completo de reincorporação de um *worker* que saiu fica como trabalho futuro, já com plano detalhado; ele reconstrói o estado retido a partir dos *deltas* já recebidos. A segunda substitui o particionamento *round-robin* atual por uma estratégia guiada pela topologia da rede, que minimize os *wires* de fronteira e, com isso, o tráfego entre *workers*. A terceira é uma interface gráfica de desktop, nos moldes do Docker Desktop e possivelmente sobre Tauri, que abriria o sistema a usuários fora da linha de comando, ampliando o público que pode reproduzir os experimentos. A quarta é mecanizar formalmente o argumento P1–P6 em um assistente de prova interativo (Coq ou Lean), o que elevaria a evidência de “verificável por testes” a “verificável por prova”, fechando o ciclo entre a teoria de Lafont e a validação empírica deste trabalho.

REFERÊNCIAS

- [1] I. Foster, C. Kesselman, and S. Tuecke, “The anatomy of the grid: Enabling scalable virtual organizations,” *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001.
- [2] H. Casanova, “Distributed computing research issues in grid computing,” *ACM SIGACT News*, vol. 33, no. 3, pp. 50–70, 2002, distributed Computing Column 8, edited by Sergio Rajsbaum.
- [3] Y. Lafont, “Interaction combinators,” *Information and Computation*, vol. 137, no. 1, pp. 69–101, 1997, article No. IC972643.
- [4] —, “Interaction nets,” in *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’90)*. New York, NY, USA: ACM, 1990, pp. 95–108.
- [5] V. Taelin, “HVM2: A parallel evaluator for interaction combinators,” HigherOrderCO, Tech. Rep., 2024, work in progress. Disponível em <https://github.com/HigherOrderCO/HVM>.
- [6] I. Mackie, “Static analysis of interaction nets for distributed implementations,” in *Proceedings of the LOMAPS Workshop on Analysis and Verification of Multiple-Agent Languages*, ser. Lecture Notes in Computer Science. Stockholm, Sweden: Springer-Verlag, 1997, pp. 217–231.
- [7] I. Mackie and J. S. Pinto, “Encoding linear logic with interaction combinators,” *Information and Computation*, vol. 176, no. 2, pp. 153–186, 2002.
- [8] D. Mazza and N. J. Ross, “Full abstraction for set-based models of the symmetric interaction combinators,” in *Foundations of Software Science and Computational Structures (FOSSACS 2012)*, ser. Lecture Notes in Computer Science, L. Birkedal, Ed., vol. 7213. Berlin, Heidelberg: Springer-Verlag, 2012, pp. 316–330.
- [9] W. Kahl, “A simple parallel implementation of interaction nets in Haskell,” in *Developments in Computational Models 2014 (DCM 2014)*, ser. Electronic Proceedings in Theoretical Computer Science, U. Dal Lago and R. Harmer, Eds., vol. 179. Open Publishing Association, 2015, pp. 33–47.
- [10] J. S. Pinto, “Sequential and concurrent abstract machines for interaction nets,” in *Proceedings of Foundations of Software Science and Computation Structures (FOSSACS 2000)*, ser. Lecture Notes in Computer Science, J. Tiuryn, Ed., vol. 1784. Springer-Verlag, 2000, pp. 267–282, on leave from Universidade do Minho.
- [11] I. Mackie and S. Sato, “Parallel evaluation of interaction nets: Some observations and examples,” 2015, work-in-progress paper.
- [12] Y. Wang, Z. Luan, D. Qian, Y. Huang, T. Chen, B. Han, Y. Ren, K. Yu, and H. Jiang, “DDGrid: A grid computing environment with massive concurrency and fault-tolerance support,” in *2008 Seventh International Conference on Grid and Cooperative Computing*. IEEE, 2008, pp. 5–14.
- [13] P. Arrighi, M. Costes, G. Dowek, and L. Maignan, “Space-time deterministic graph rewriting,” *arXiv preprint arXiv:2404.05838v2*, October 2024, submitted to LIPICs. Université Paris-Saclay, Inria, CNRS, LMF / Univ Paris Est Creteil, LACL.